PASSENGER SURVIVAL PREDICTION USING MACHINE LEARNING

A Project Report

Submitted in partial fulfilment of the requirements for the
Machine Learning Course

Submitted by

SRINITHI S

11523040487

Department of CSE

School of Engineering and Technology

Dhanalakshmi Srinivasan University

Academic Year 2026 – 2027

ABSTRACT

Predicting passenger survival is a classic binary classification problem in machine learning, useful for demonstrating how demographic and travel-related attributes relate to survival outcomes under crisis conditions. This project uses the Titanic dataset, containing 891 passenger records with attributes such as class, sex, age, fare, and family size, alongside a binary survival label.

The dataset was cleaned by removing three low-value columns (PassengerId, Name, Ticket) and one column with excessive missing data (Cabin, 687 of 891 values missing). Missing Age values (177) were filled with the median age, and missing Embarked values (2) were filled with the most frequent port. Categorical columns (Sex, Embarked) were encoded numerically. The cleaned data was split into training (712 records) and testing (179 records) sets using an 80:20 stratified split, and a Logistic Regression classifier was trained and evaluated.

The model achieved an accuracy of 80.45%, precision of 79.31%, recall of 66.67%, and an F1-score of 72.44%. These results show the model reliably distinguishes survivors from non-survivors, with a stronger ability to correctly identify non-survivors than survivors.

Keywords: Machine Learning, Classification, Titanic Dataset, Logistic Regression, Data Preprocessing.

TABLE OF CONTENTS

1. Problem Definition and Objectives

2. Dataset Selection, Preprocessing, and Feature Engineering

3. Model Selection, Implementation, and Justification

4. Results, Evaluation Metrics, and Discussion

5. Conclusion and Future Work

References

Appendix A — Glossary of Terms

Appendix B — Full Implementation Code

1. PROBLEM DEFINITION AND OBJECTIVES

1.1 Background

Predicting outcomes from historical data is a common application of supervised machine learning. The sinking of the Titanic in 1912 provides a well-documented dataset where the survival of each passenger is known, alongside attributes describing who they were and how they traveled. This makes it a suitable dataset for demonstrating a complete binary classification pipeline.

1.2 Problem Definition

The problem addressed in this project is the binary classification of passengers into two categories — survived or did not survive — based on attributes available at the time of boarding (class, sex, age, family size, fare paid, and port of embarkation). The input is a passenger's feature vector, and the output is one of two classes: 0 (did not survive) or 1 (survived).

1.3 Objectives

- To select a labeled dataset suitable for supervised classification.
- To inspect the dataset for missing values and irrelevant columns.
- To preprocess and clean the dataset, handling missing values and encoding categorical features.
- To split the cleaned dataset into training and testing subsets.
- To train a Logistic Regression classifier on the training data.
- To evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.

1.4 Scope of the Project

The scope of this project is limited to the attributes present in the original dataset. External factors not recorded in the dataset — such as physical swimming ability or exact cabin location relative to lifeboats — are outside the scope. The project is an academic exercise demonstrating the full classification workflow, not a production prediction system.

1.5 Proposed Solution

The proposed solution applies a sequence of preprocessing and machine learning operations. Raw passenger data is cleaned, missing values are filled, and categorical attributes are converted into numeric form. The resulting feature set is used to train a Logistic Regression classifier, which predicts whether a passenger survived, and standard evaluation metrics measure the effectiveness of the model.

1.6 Stakeholders and Use Cases

Several groups benefit from this kind of survival-prediction pipeline. Students and researchers use it as an accessible introduction to supervised classification on real historical data. Data science educators use the Titanic dataset as a standard benchmark for teaching preprocessing and evaluation techniques. More broadly, the same modeling approach — predicting a binary outcome from structured demographic and situational attributes — generalizes to use cases such as insurance risk scoring and emergency-response planning.

1.7 Assumptions and Constraints

The project makes several simplifying assumptions consistent with its academic scope. It is assumed that the recorded attributes (class, sex, age, family size, fare, embarkation port) are sufficient to capture a meaningful share of the pattern behind survival, even though many real determinants — such as exact cabin location or physical ability — are not recorded in the dataset.

Several constraints also shape the project. The dataset is limited to 891 records, which restricts how complex a model can be trained without overfitting; this is part of why a simple, interpretable Logistic Regression model was chosen over a higher-capacity algorithm. The dataset also reflects a single historical event, which may limit how well any learned pattern generalizes beyond it.

1.8 Methodological Approach

The project follows a standard applied machine learning methodology consisting of problem framing, data acquisition, data cleaning, feature engineering, model selection, training, evaluation, and interpretation of results. Each of these stages is treated as a distinct and documented step so that the overall workflow remains transparent and reproducible.

2. DATASET SELECTION, PREPROCESSING, AND FEATURE ENGINEERING

2.1 Dataset Selection

The project uses the Titanic dataset, a widely used publicly available dataset for classification tasks. Each record represents one passenger, described by demographic and travel attributes, along with a binary survival label.

2.2 Dataset Characteristics

The original dataset contained 891 records across 12 columns. Initial inspection using df.info() and df.isnull().sum() revealed the following missing values:

Column	Missing Values
Age	177
Cabin	687
Embarked	2
All other columns	0

2.3 Data Preprocessing

- Column removal: PassengerId (row index with no predictive value), Name and Ticket (free-text identifiers with no direct predictive pattern), and Cabin (77% missing, too sparse to reliably impute) were dropped.
- Missing value handling: Age was filled with the column median; Embarked was filled with its most frequent value (mode).
- Categorical encoding: Sex was mapped to numeric values (male = 0, female = 1). Embarked was mapped to numeric values (S = 0, C = 1, Q = 2).

After preprocessing, the dataset contained 8 columns (Survived, Pclass, Sex, Age, SibSp, Parch, Fare, Embarked) with zero missing values.

Preprocessing Step	Result
Initial columns	12
Columns dropped	4
Final columns	8
Missing values after cleaning	0

2.4 Feature Engineering

No additional derived features were engineered beyond encoding the two categorical columns. All retained features (Pclass, Sex, Age, SibSp, Parch, Fare, Embarked) were used directly as numeric inputs, since Logistic Regression does not strictly require feature scaling for correctness.

2.5 Data Splitting Strategy

A stratified 80:20 train-test split was used, producing 712 training records and 179 testing records. Stratification preserves the proportion of survivors to non-survivors in both subsets. A fixed random state of 42 was used to ensure the split is reproducible.

3. MODEL SELECTION, IMPLEMENTATION, AND JUSTIFICATION

3.1 Model Selection: Logistic Regression

Logistic Regression was selected as the classification algorithm. It is a standard, interpretable model for binary classification on numeric tabular data, estimating the probability that a given passenger survived based on a weighted combination of input features.

3.2 Justification for Model Selection

- Well suited to binary classification with numeric features.
- Highly interpretable — each coefficient shows the direction and strength of a feature's effect on survival.
- Computationally efficient and unlikely to overfit on a dataset of this size.
- Provides a strong, standard baseline for tabular classification problems.

3.3 Alternative Models Considered

Before finalizing Logistic Regression, several alternative algorithms commonly used for tabular classification were considered.

Algorithm	Strengths	Reason Not Selected
Decision Tree	Handles non-linear relationships	More prone to overfitting on this dataset size; less directly interpretable
Random Forest	Strong general-purpose performance	Higher computational cost; less interpretable than a single linear model
Support Vector Machine	Effective in high-dimensional spaces	Less straightforward probability outputs; unnecessary complexity for this feature set
Logistic Regression	Simple, interpretable, well suited to binary numeric classification	Selected as the final model

3.4 Hyperparameters and Configuration

The Logistic Regression classifier was used with Scikit-learn's default hyperparameters (L2 regularization, C = 1.0), with the maximum iteration limit increased to 1000 (max_iter = 1000) to ensure the solver fully converges. No extensive hyperparameter search was performed in this iteration of the project, as the objective was to establish a solid baseline; hyperparameter tuning is identified as an item for future work in Chapter 5.

3.5 Implementation Details

The system was implemented in Python using a single script (main.py), executed locally via the terminal. Key libraries included Pandas for dataset loading and cleaning, and Scikit-learn for train-test splitting, model training, and evaluation.

The dataset was loaded using Pandas' read_csv function. Missing-value checks were performed with isnull().sum(), followed by column removal, missing-value imputation, and categorical encoding. The cleaned features were split into training and testing sets, a LogisticRegression classifier was fitted on the training data, and predictions on the test set were compared against actual labels to compute evaluation metrics.

3.6 Key Implementation Code

Dataset loading and inspection:

```
import pandas as pd
df = pd.read_csv('titanic.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())
```

Preprocessing:

```
df = df.drop(columns=['PassengerId','Name','Ticket','Cabin'])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
```

Train-test split:

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=['Survived'])
y = df['Survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

Model training and evaluation:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1:', f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

4. RESULTS, EVALUATION METRICS, AND DISCUSSION

4.1 Evaluation Metrics

Metric	Result
Accuracy	80.45%
Precision	79.31%
Recall	66.67%
F1-score	72.44%

4.2 Confusion Matrix

```
[[98 12]
 [23 46]]
```

- True Negatives (correctly predicted did-not-survive): 98
- False Positives (predicted survived, actually did not): 12
- False Negatives (predicted did-not-survive, actually survived): 23
- True Positives (correctly predicted survived): 46

4.3 Interpretation of Results

Accuracy of 80.45% indicates the model correctly classified the large majority of test passengers. Precision of 79.31% means that of all passengers the model predicted as survivors, about 4 in 5 actually survived. Recall of 66.67% indicates the model correctly identified two-thirds of actual survivors, missing the remaining third (23 false negatives). The F1-score of 72.44%, the harmonic mean of precision and recall, reflects this moderate imbalance between the two metrics.

4.4 Error Analysis

The model's errors on the test set are a mix of 12 false positives (non-survivors incorrectly predicted as survivors) and 23 false negatives (survivors incorrectly predicted as non-survivors), with false negatives more than double the false positives. This asymmetry is consistent with recall (66.67%) being noticeably lower than precision (79.31%): the model is somewhat conservative, more often failing to identify an actual survivor than incorrectly flagging a non-survivor as one.

4.5 Discussion

The gap between precision (79.31%) and recall (66.67%) shows the model is more conservative in predicting survival than it should ideally be — it correctly rules out most non-survivors but misses a meaningful share of actual survivors. This is consistent with the class imbalance in the data itself (549 non-survivors vs. 342 survivors in the original dataset), which can bias a linear model toward the majority class.

4.6 Comparison with a Naive Baseline

To contextualize the model's performance, it is useful to compare it against a trivial baseline that always predicts the majority class (did not survive). Given the original class distribution (549 non-survivors out of 891 total passengers), such a baseline would achieve an accuracy of approximately 61.62%, with 0% recall for the survivor class. The trained Logistic Regression model substantially outperforms this baseline, achieving 80.45% accuracy while also correctly identifying two-thirds of actual survivors, confirming that the model has learned genuine discriminative patterns rather than relying on the class imbalance alone.

4.7 Strengths

- Simple, fast, and fully reproducible pipeline.
- High overall accuracy on unseen test data.
- Coefficients are directly interpretable in terms of real-world factors (class, sex, age).

4.8 Limitations

- Recall (66.67%) shows a meaningful portion of actual survivors were missed.
- No feature engineering beyond basic encoding was applied (e.g., family size was not combined from SibSp and Parch).
- The dataset reflects one historical event and may not generalize to other survival-prediction contexts.

4.9 Practical Applications

The approach demonstrated in this project extends to any setting requiring binary risk or outcome classification from structured attributes — for example, insurance risk assessment, emergency-response prioritization, and educational tools for teaching supervised classification workflows.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project developed a supervised machine learning pipeline to classify Titanic passengers as survivors or non-survivors. After removing four low-value or excessively sparse columns and handling the remaining missing values in Age and Embarked, a Logistic Regression classifier was trained on an 80:20 stratified split of the cleaned dataset. The model achieved 80.45% accuracy, 79.31% precision, 66.67% recall, and an F1-score of 72.44%, demonstrating that a simple, interpretable linear model can effectively capture the relationship between passenger attributes and survival outcome.

5.2 Future Scope

- Engineer a combined "family size" feature from SibSp and Parch.
- Apply feature scaling and compare against regularized variants (Ridge/Lasso logistic regression).
- Compare against non-linear models such as Random Forest to assess whether recall improves.
- Apply k-fold cross-validation for a more robust performance estimate.

References

1. Data Science Dojo. Titanic Dataset. GitHub repository.

2. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.

APPENDIX A — GLOSSARY OF TERMS

Term	Definition
Survived	Target label: 1 if the passenger survived, 0 if not.
Pclass	Ticket class (1st, 2nd, or 3rd), used as a proxy for socio-economic status.
SibSp	Number of siblings or spouses the passenger had aboard.
Parch	Number of parents or children the passenger had aboard.
Logistic Regression	A probabilistic linear model used for binary classification.
Precision	The proportion of predicted-positive instances that are actually positive.
Recall	The proportion of actual positive instances that are correctly predicted as positive.
F1-score	The harmonic mean of precision and recall.
Stratified split	A train-test split that preserves the class proportions of the original dataset.
Confusion matrix	A table summarizing true/false positive and negative predictions.

APPENDIX B — FULL IMPLEMENTATION CODE

This appendix consolidates the full code used across the project pipeline, from data loading through model evaluation, for reference and reproducibility.

```
import pandas as pd
df = pd.read_csv('titanic.csv')
print(df.head())
print(df.info())
print(df.isnull().sum())

df = df.drop(columns=['PassengerId','Name','Ticket','Cabin'])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
df['Embarked'] = df['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})

from sklearn.model_selection import train_test_split
X = df.drop(columns=['Survived'])
y = df['Survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1:', f1_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```